

Aula 4: Aprendendo a rede — Parte 2: retropropagação

Regra da cadeia no grafo computacional • autograd • PyTorch

Prof. Marcelo Keese Albertini

GBC073 — Inteligência Computacional • Universidade Federal de Uberlândia

2026



- ▶ **O problema do crédito:** quem é responsável por cada erro?

- ▶ **O problema do crédito:** quem é responsável por cada erro?
- ▶ **A regra da cadeia no grafo computacional:** o erro viaja para trás

- ▶ **O problema do crédito:** quem é responsável por cada erro?
- ▶ **A regra da cadeia no grafo computacional:** o erro viaja para trás
- ▶ **A regra delta generalizada:** a atualização de cada peso

- ▶ **O problema do crédito:** quem é responsável por cada erro?
- ▶ **A regra da cadeia no grafo computacional:** o erro viaja para trás
- ▶ **A regra delta generalizada:** a atualização de cada peso
- ▶ **Autograd:** a implementação em PyTorch

PARTE 1

O problema do crédito



- ▶ Rede = composição de camadas: $f = f_L \circ \dots \circ f_1$
- ▶ Erro medido na saída E : culpa *compartilhada* de todos os pesos
- ▶ Atribuir a cada w_{ji} a sua parcela de responsabilidade
- ▶ Regra do perceptron só alcança a *última* camada — ocultas sem sinal de aprendizado
- ▶ Retropropagação resolve a atribuição de crédito: gradiente camada a camada

PARTE 2

A regra da cadeia no grafo computacional



- ▶ Rede como grafo computacional: nós = operações, arestas = tensores
- ▶ Gradientes locais de cada nó, encadeados pela regra da cadeia
- ▶ Propaga para trás a *sensibilidade* do erro a cada ativação
- ▶ Custo da retropropagação: comparável ao da propagação adiante

PARTE 3

A regra delta generalizada



- ▶ O erro retropropagado na camada j : $\delta_j = \varphi'(v_j) \sum_k \delta_k w_{kj}$.
- ▶ Atualização: $\Delta w_{ji} = \eta \delta_j y_i$.
- ▶ A regra delta (Widrow–Hoff) é o caso de uma camada só.
- ▶ Funções de ativação diferenciáveis viram requisito — adeus degrau.

PARTE 4

Autograd e a implementação em PyTorch



- ▶ Diferenciação automática: grafo computacional registrado durante o *forward* (`tensor.requires_grad`)
- ▶ `loss.backward()` percorre o grafo e acumula gradientes
- ▶ `optimizer.step()` aplica a atualização; `zero_grad()` entre passos
- ▶ Não se escreve retropropagação à mão: escreve-se o *forward* e o framework deriva

Intuição

Retropropagação = regra da cadeia de trás para frente — o que viaja para trás é a sensibilidade do erro a cada peso.